

처음 만나는 MSA

쇼핑몰 주문 시스템으로 배우는 마이크로서비스 아키텍처 들어가며

어느 날 팀장이 이런 말을 합니다.

"결제 로직 버그 잡았지? 빠르게 배포해."

그런데 배포 버튼을 누르는 순간, 심장이 두근거립니다. 결제만 고쳤는데 왜인지 회원 기능, 상품 목록, 주문 내역까지 전부 잠깐 멈춥니다. 배포할 때마다 전체 서비스가 재시작되기 때문입니다. 팀원들은 배포 공지를 올리고, 사용자들은 잠깐의 장애를 경험합니다.

이것이 모든 기능이 하나로 구성된 **모놀리식(Monolithic)** 구조의 현실입니다.

서비스가 커질수록 이 두려움은 커집니다. 코드 한 줄 바꾸기가 무섭고, 팀원이 늘어날수록 충돌이 잦아집니다.

마이크로서비스 아키텍처(MSA)는 바로 이 문제를 풀기 위해 등장했습니다.

하지만 MSA를 처음 접하면 용어들이 낯섭니다. 분산 트랜잭션, Saga 패턴, 이벤트 기반 아키텍처 — 읽을 때는 이해한 것 같아도, 막상 코드를 작성하려면 손이 멈춥니다. **직접 만들어보아야** 비로소 왜 이렇게 구성하는지 체감할 수 있습니다.

그래서 이 책은 쇼핑몰 주문 서비스를 함께 만들면서 MSA의 전체 그림을 익힙니다. 처음에는 가장 단순한 동기식 구조로 시작합니다. 그 한계를 직접 느끼고 나서, 클린 아키텍처로 코드를 개선하고, Kafka로 비동기 통신을 도입하고, 마지막으로 WebSocket 실시간 알림까지 붙입니다. 하나의 프로젝트가 장을 거듭하며 진화하는 여정입니다.

이 책에서 만드는 것

각 장은 이전 장의 한계를 느끼고, 그것을 해결하는 방식으로 이어집니다.

장	핵심 기술	이전 장의 어떤 한계를 해결하는가
2장	REST + 보상 트랜잭션	동기식 MSA의 기본 구조
3장	Clean Architecture + Kubernetes	코드 결합도 + 로컬에서만 실행되는 한계
4장	Apache Kafka + Orchestration Saga	서비스 간 동기 결합
5장	WebSocket (STOMP)	폴링으로 인한 비효율 + 자동 배송 완료 처리

책의 마지막에는 아래와 같은 시스템이 완성됩니다.

```
sequenceDiagram
    autonumber
    actor User as 사용자
```

```
participant Order as 주문 서비스
participant Orch as 오케스트레이터 (조율자)
participant Product as 상품 서비스
participant Delivery as 배달 서비스
actor Driver as 배달 기사

note over User, Order: 1. 주문 시작
User->>Order: 주문 생성 요청
Order-->>Orch: 이벤트: 주문 생성됨

note over Orch, Product: 2. 재고 처리
Orch->>Product: 명령: 재고 차감해라
Product-->>Orch: 이벤트: 재고 차감 완료됨

note over Orch, Delivery: 3. 배달 처리
Orch->>Delivery: 명령: 배달 생성해라

note over Delivery, Driver: 4. 배달 완료
Driver->>Delivery: 배달 완료 처리 (앱에서 터치)
Delivery-->>Orch: 이벤트: 배달 완료됨

note over Orch, User: 5. 주문 최종 완료 및 알림
Orch->>Order: 명령: 주문 최종 완료 처리해라
Order-->>User: 실시간 알림: "주문이 완료되었습니다!"
```

개발 환경 준비

실습을 시작하기 전에 아래 도구를 설치해주세요.

도구	버전	용도
Java	21	애플리케이션 실행
IntelliJ IDEA	최신 버전	코드 작성
Docker Desktop	최신 버전	컨테이너 실행
Kubernetes	Docker Desktop 내장	K8s 실습 (3장~)

Docker Desktop을 설치하면 Kubernetes를 함께 활성화할 수 있습니다. Docker Desktop → Settings → Kubernetes → Enable Kubernetes 체크 후 Apply & Restart합니다.

Gradle과 Spring Boot는 프로젝트 내부에 포함되어 있으므로 별도 설치가 필요 없습니다.

1장. MSA란 무엇인가?

목표

이 장이 끝나면 다음을 이해하게 됩니다.

- 모놀리식 아키텍처가 왜 한계에 부딪히는지
- 마이크로서비스가 어떤 방식으로 그 문제를 해결하는지
- 우리가 만들 시스템이 어떻게 구성되는지
- MSA의 핵심 과제인 분산 트랜잭션이 무엇인지
- 이 책의 2장~5장이 어떤 흐름으로 이어지는지

1.1 쇼핑몰을 하나의 서버로 만들면 어떻게 될까?

처음에는 아무 문제가 없었다

백화점을 떠올려 보세요. 수십 개의 매장이 한 건물 안에 모여 있습니다. 고객은 한 곳에서 모든 것을 해결할 수 있고, 운영사 입장에서는 전기·냉방·보안·고객 데이터를 한 곳에서 통합 관리합니다. 이 구조는 단순하고 효율적입니다.



소프트웨어도 같은 이야기입니다. 처음에는 하나의 서버에 모든 기능을 넣는 **모놀리식** 구조가 단순하고 빠릅니다.

성장하면서 균열이 생긴다

백화점이 잘 돼서 방문객이 열 배로 늘었습니다. 이제 문제가 보이기 시작합니다.

[백화점의 한계]

문제1: 한 매장에서 화재 발생

→ 스프링클러 작동·대피령으로 전 층 영업 중단

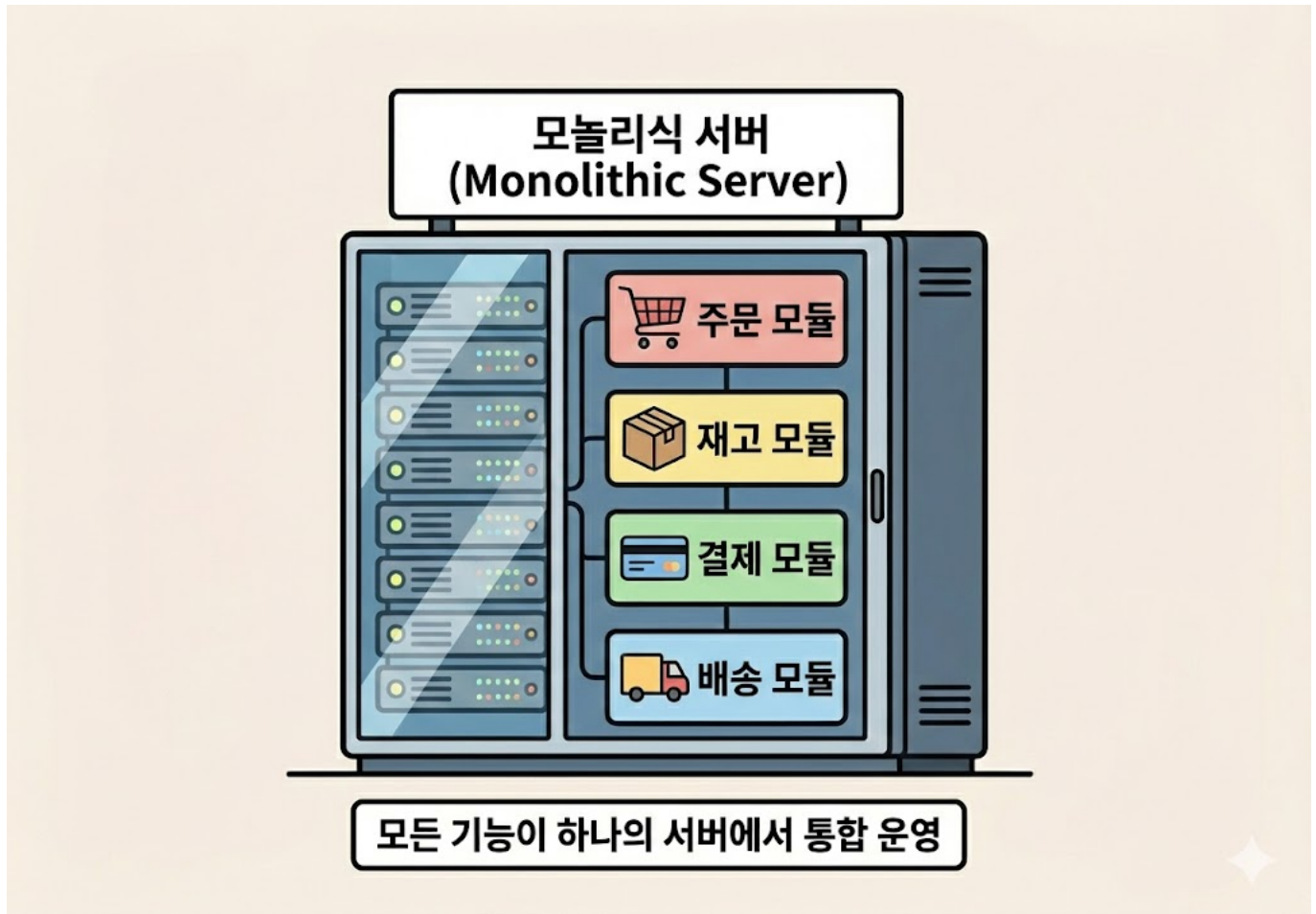
문제2: 전자제품 세일로 3층에 사람이 몰림

→ 3층만 확장 불가, 건물 전체를 새로 지어야 함

문제3: 건물 전기 배선 전면 교체

→ 공사 기간 동안 전 층 영업 중단

소프트웨어 세계에서든 똑같은 일이 벌어집니다. 회원 10만 명, 하루 주문 1만 건이 되었을 때, 모놀리식 구조의 균열이 드러납니다.



배포가 두렵다 — 건물 전기 배선을 교체하면 전 층이 일제히 문을 닫아야 하듯, 주문 기능 하나를 수정해도 배포 시 회원·상품·배송까지 전부 재시작해야 합니다.

장애가 전파된다 — 한 매장 화재에 스프링클러가 작동하면 전 층이 영업을 멈춰야 하듯, 배송 기능 버그로 서버가 느려지면 배송과 무관한 회원 로그인도 함께 느려집니다.

확장이 어렵다 — 전자제품 층만 늘릴 수 없듯, 블랙프라이데이에 주문 기능만 서버를 늘리고 싶어도 모놀리식에서는 전체 서버를 복제해야 합니다.

팀이 커지면 충돌이 잦아진다 — 한 건물 안에서 여러 팀이 동시에 인테리어를 바꾸면 서로 방해가 되듯, 10명이 하나의 코드베이스를 동시에 수정하면 하루에도 수십 번 충돌이 발생합니다.

1.2 마이크로서비스: 역할을 나눈다

백화점 vs 개별 상점

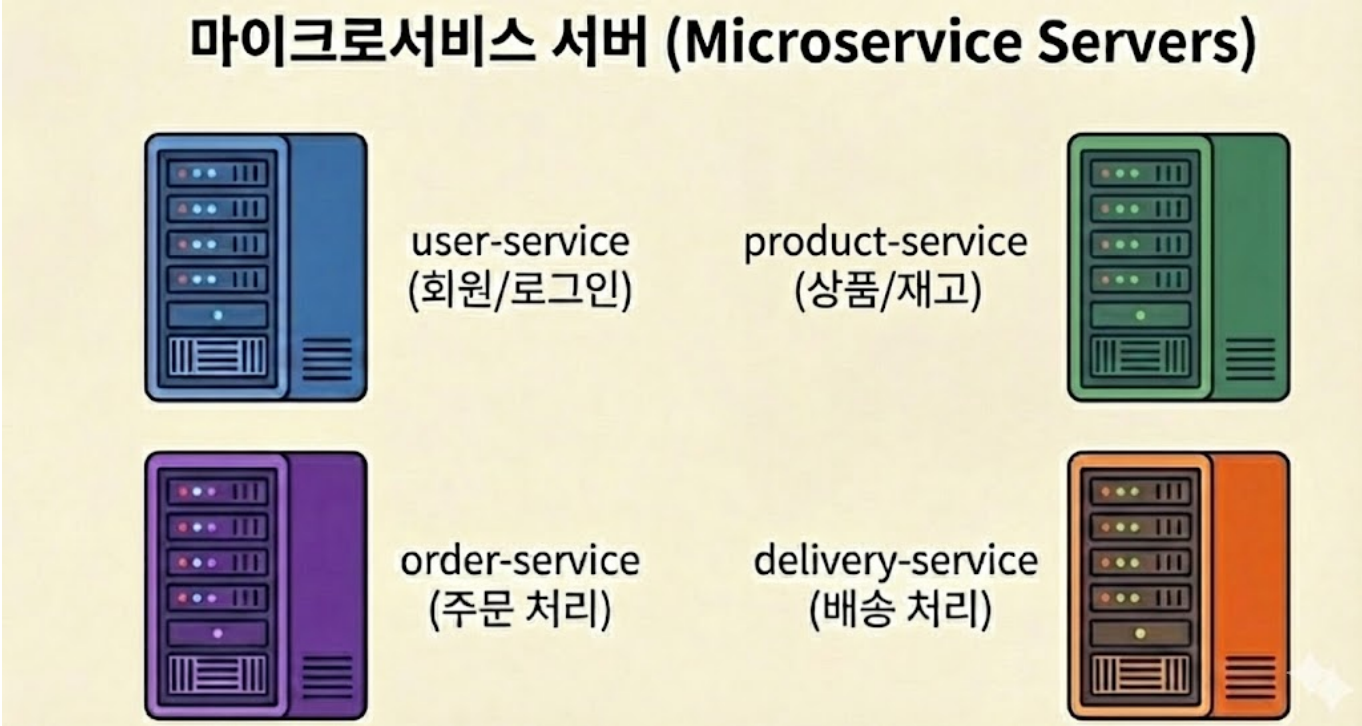
개별 상점 방식은 백화점과 구조 자체가 다릅니다. 각 매장이 독립된 건물로 운영되어, 자신만의 전기·냉방·입구를 가집니다.



백화점 구조와 비교하면 차이가 분명합니다.

백화점		개별 상점
한 매장 화재 발생	스프링클러·대피령으로 전 층 영업 중단	해당 매장만 소방 처리, 나머지 정상 영업
전자제품 수요 폭증	건물 전체를 확장해야 함	전자제품점만 확장
건물 전기 배선 교체	공사 기간 동안 전 층 영업 중단	해당 매장만 임시 폐쇄, 나머지 정상 영업

마이크로서비스 아키텍처가 바로 이 방식입니다. 하나의 큰 서버 대신, 기능별로 작은 서비스들을 분리합니다.



이제 각 서비스는 독립적으로 배포하고, 독립적으로 확장할 수 있습니다. 주문 서비스에 버그가 생겨도 회원 서비스는 영향을 받지 않습니다.

MSA vs 모놀리식

특성	모놀리식	마이크로서비스
배포	전체 재배포	해당 서비스만 배포
장애 격리	전체 영향	해당 서비스만 영향
확장	전체 복제	필요한 서비스만 확장
팀 분리	하나의 코드베이스	서비스별 독립 개발

1.3 우리가 만들 시스템 설계

문제를 이해했으니, 이제 직접 만들어볼 시스템을 설계해보겠습니다. 이 책의 쇼핑몰 주문 시스템은 4개의 마이크로서비스로 구성됩니다.

서비스	포트	역할
user-service	8083	로그인, JWT 발급, 사용자 조회
product-service	8082	상품 목록, 재고 조회 및 증감
order-service	8081	주문 생성·조회·취소 (핵심)
delivery-service	8084	배송 생성·조회·취소

4장부터는 **orchestrator**(8085)가 추가되어 Kafka를 통한 비동기 워크플로우를 조율합니다.

2~3장 아키텍처 (동기 통신)

2장과 3장에서 만들 시스템입니다. order-service가 중심에서 다른 서비스를 직접 REST로 호출합니다.

```
sequenceDiagram
    autonumber
    actor Client as 클라이언트
    participant User as user-service :8083
    participant Order as order-service :8081
    participant Product as product-service :8082
    participant Delivery as delivery-service :8084

    Client->>User: POST 로그인
    User-->>Client: JWT 토큰 발급

    Client->>Order: POST 주문 생성 (JWT)
    Order->>Product: 재고 감소
    Product-->>Order: 응답
    Order->>Delivery: 배송 생성
    Delivery-->>Order: 응답
    Order-->>Client: 주문 완료
```

사용자가 주문을 생성하면 order-service가 product-service와 delivery-service를 차례로 호출합니다. 이 구조를 먼저 직접 구현해보면, 나중에 비동기 방식으로 전환했을 때 그 차이가 더 명확하게 느껴집니다.

기술 스택

항목	버전
Java	21
Spring Boot	4.0.1
Gradle	8.14
JWT	com.auth0:java-jwt:4.4.0
DB (개발)	H2 in-memory
DB (운영)	MySQL (3장~)
Kafka	spring-boot-starter-kafka (4장~)
WebSocket	spring-boot-starter-websocket (5장~)

1.4 MSA의 핵심 과제: 분산 트랜잭션

서비스를 분리하면 좋은 점이 많지만, 동시에 새로운 문제가 생깁니다. 바로 **분산 트랜잭션**입니다. 이 개념을 이해하는 것이 이 책의 핵심입니다.

모놀리식에서는 쉬웠던 것이

모놀리식에서는 데이터베이스 트랜잭션이 간단합니다. 주문, 재고 변경, 배송 생성을 하나의 `@Transactional` 블록 안에 놓으면, 하나라도 실패하면 전부 자동 롤백됩니다.

[참고] 동작 이해용입니다. 모놀리식에서 단일 트랜잭션으로 처리하는 예시입니다.

```
// 모놀리식: 하나의 트랜잭션으로 처리 가능
@Transactional
public void createOrder() {
    decreaseStock(); // 재고 감소
    createDelivery(); // 배송 생성
    saveOrder();      // 주문 저장
    // 실패 시 세 가지 모두 자동 롤백
}
```

MSA에서는 불가능하다

하지만 MSA에서는 각 서비스가 **독립된 데이터베이스**를 가집니다. product-service의 DB와 delivery-service의 DB가 물리적으로 분리되어 있습니다. 하나의 트랜잭션으로 묶을 방법이 없습니다.

```
sequenceDiagram
    participant O as order-service
    participant P as product-service
    participant D as delivery-service

    O->>P: 재고 감소 요청
    P-->>O: 성공 응답
    O->>D: 배송 생성 요청
    D-->>O: 실패 응답
    Note over O: 재고는 이미 줄어있는데...
    어떻게 복구하지?
```

배송 생성이 실패했을 때, 이미 감소된 재고를 어떻게 되돌릴까요? product-service와 delivery-service의 DB가 분리되어 있으므로, 자동 롤백이 불가능합니다.



해결 방법: Saga 패턴

이 책에서는 이 문제를 **Saga 패턴**으로 해결합니다. 두 가지 방식을 순서대로 배웁니다.

Choreography Saga(보상 트랜잭션) — 2장, 3장

실패가 발생하면 이전 단계에 직접 복구를 요청합니다. order-service가 직접 product-service에 "재고 돌려줘"라고 요청합니다.

```
sequenceDiagram
    participant O as order-service
    participant P as product-service
    participant D as delivery-service

    O->>P: "재고 줄여"
    P-->>O: 성공
    O->>D: "배송 만들어"
    D-->>O: 성공
    Note over O: 주문 완료
```

```
sequenceDiagram
    participant O as order-service
    participant P as product-service
    participant D as delivery-service

    O->>P: "재고 줄여"
    P-->>O: 성공
    O->>D: "배송 만들어"
    D-->>O: 실패!
    O->>P: "재고 돌려줘" (보상)
    Note over O: 주문 취소
```

서비스끼리 서로 직접 호출해서 복구하는 방식입니다. 단순하지만, 서비스 수가 늘어날수록 복잡해집니다.

Orchestration Saga — 4장, 5장

별도의 orchestrator가 전체 흐름을 조율합니다. 각 서비스는 자신의 일만 하고 Kafka로 결과를 발행합니다. orchestrator가 결과를 받아 다음 단계를 결정합니다.

```
sequenceDiagram
    participant Orch as orchestrator
    participant P as product-service
    participant D as delivery-service
    participant O as order-service

    Orch->>P: ① "재고 줄여"
    P-->>Orch: 성공
    Orch->>D: ② "배송 만들어"
    D-->>Orch: 성공
    Orch->>O: ③ "주문 완료해"
```

```
sequenceDiagram
    participant Orch as orchestrator
    participant P as product-service
    participant D as delivery-service
    participant O as order-service

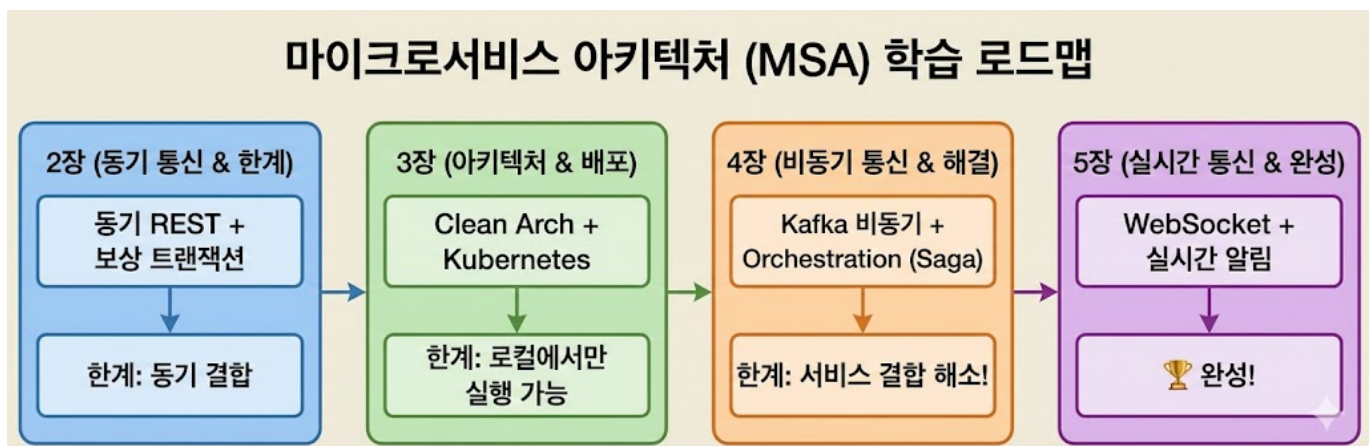
    Orch->>P: ① "재고 줄여"
    P-->>Orch: 성공
    Orch->>D: ② "배송 만들어"
    D-->>Orch: 실패!
    Orch->>P: ③ "재고 돌려줘" (자동 롤백)
    Orch->>O: "주문 취소해"
```

orchestrator가 전체 흐름을 알고 있기 때문에, 실패 시 자동으로 롤백 명령을 내립니다.

두 방식을 모두 직접 구현해보면, 각각의 장단점이 자연스럽게 체감됩니다.

1.5 이 책의 학습 흐름

이제 전체 그림이 보입니다. 이 책은 하나의 시스템이 단계별로 진화하는 여정입니다. 각 장은 이전 장의 한계를 느끼는 것에서 시작합니다.



2장 — 4개 서비스를 REST로 연결하고 보상 트랜잭션을 구현합니다. MSA의 뼈대를 직접 손으로 만드는 장입니다.

3장 — 2장 코드의 아쉬운 점을 클린 아키텍처로 개선하고, Kubernetes에 올려 운영 환경을 경험합니다.

4장 — 동기 REST 호출의 한계를 Kafka로 해결합니다. 서비스가 완전히 분리되는 경험을 합니다.

5장 — 배달 기사가 완료 API를 호출하는 순간, 사용자 화면에 실시간 알림이 뜨는 시스템을 완성합니다.

정리

- **모놀리식**은 처음에는 단순하지만, 서비스가 커지면 배포·장애·확장 문제가 생긴다.
- **마이크로서비스**는 기능별로 서비스를 분리하여 각자 독립적으로 배포하고 확장할 수 있게 한다.
- MSA의 핵심 과제는 **분산 트랜잭션**이다. 각 서비스의 DB가 분리되어 있어 단일 트랜잭션으로 묶을 수 없다.

- **Saga 패턴**으로 분산 트랜잭션을 해결한다. 보상 트랜잭션(2~3장)과 Orchestration Saga(4~5장) 두 가지를 이 책에서 배운다.
- 이 책은 하나의 쇼핑몰 주문 시스템이 2장부터 5장까지 단계적으로 진화하는 이야기다.

이제 직접 코드를 작성할 시간입니다. 2장에서는 4개의 서비스를 REST로 연결하고, 보상 트랜잭션을 구현합니다. 처음에는 단순하게 — 그 단순함이 나중에 어떤 문제를 만드는지를 직접 느끼는 것이 목표입니다.